

PROJECT OVERVIEW

TVM-based abstraction layer to simplify deploying AI models on TI's AM69A.

Runs ONNX models across ARM, C7x DSP, and MMA via unified Python interface

Focus: Architecture Design and Prototype Validation

THE CHALLENGE

- Support AI models across multiple processors without burdening developers
- Keep parsing, compiling, and execution simple and consistent
- Abstract limited hardware access and complex toolchains

WHAT IS TVM?

An open-source compiler that takes an ONNX model and translates it into code that can run efficiently on specific hardware

WHAT WE BUILT

- ✓ Python API (deploy, run, set_backend)
- ✓ C++ bridge via pybind11
- ✓ ONNX Runtime backend integration
- ✓ End-to-end demo with ONNX model

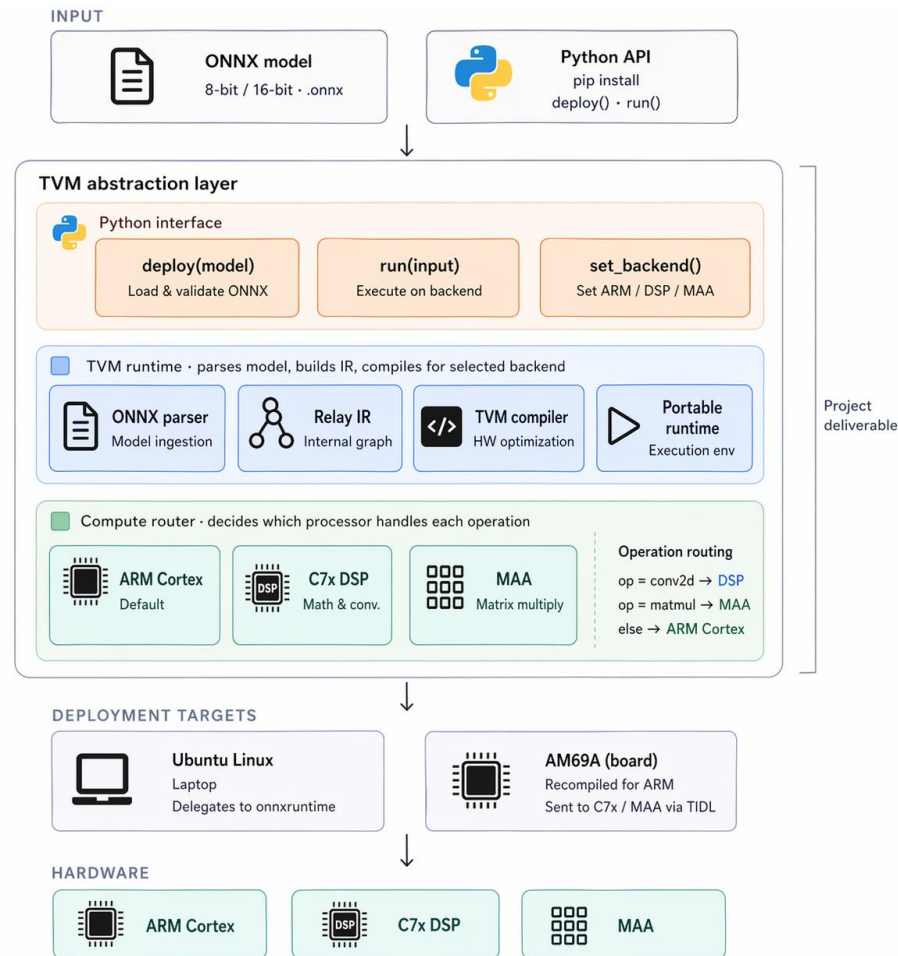
```
from tvm_onnx_passthrough import TVMModule

m = TVMModule("model.onnx")

output = m.run({"x": input_array})
```

ARCHITECTURE DESIGN

Input (ONNX + Python API) → Abstraction Layer → Runtime/Compiler → Hardware Routing → Deployment



TECH STACK



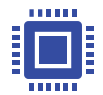
Python 3
Interface



C++
Pybind11 Bridge



ONNX
Runtime Backend



ARM/C7x/MMA
Targets

BUSINESS VALUE

- **Faster Deployment** – Unified interface reduces development time
- **Automated Execution & Routing** – Handles parsing, compilation, and hardware mapping
- **Simplified API** – Single Python interface for all users
- **Enables Adoption** – SDK + docs make integration easy
- **Scalable Foundation** – Supports optimization, benchmarking, and future hardware

SDK (Built & Documented)

- ✓ Environment setup & build pipeline established
- ✓ Install & setup instructions
- ✓ TI sample apps identified for validation (C7x path)
- ✓ Design docs for abstraction layer

FUTURE WORK

- Full TVM compiler/runtime layer (replace ONNX Runtime)
- Compute routing logic across ARM, C7x, DSP, MMA
- Validate on AM69A hardware
- Benchmarking & tuning